Design and Validation of a Thrust Vector Controlled Model Rocket Using PID-Kalman Architecture

Marc Julian Dackiw[a]

[a]Flint Hill School, 3320 Jermantown Road, Oakton, VA 22124, USA

ORCID: 0009-0007-9183-1596

Corresponding author: Marc Julian Dackiw

Email: mjdackiw@gmail.com

Telephone: [+1 (571)-730-0528] Fax: [none]

Full school postal address: Flint Hill School, 3320 Jermantown Road, Oakton, VA 22124, USA

Abstract

This paper presents the design, implementation, and simulation-based validation of a thrust vector controlled model rocket. The system uses a custom two-axis gimbal to deflect motor thrust in real time, replacing passive aerodynamic fins as the sole means of attitude stabilization. A closed-loop PID controller continuously corrects pitch and yaw deviations, while a Kalman filter fuses gyroscope and accelerometer data to produce reliable orientation estimates under vibration and high-acceleration conditions. The flight software is structured as an autonomous state machine designed to manage all phases of flight, from launch detection through apogee identification and recovery. Because powered flight testing was not possible due to airspace restrictions, system performance was evaluated entirely through physics-based simulation, including disturbance rejection analysis and a two-axis Monte Carlo study. These simulations validate the control architecture, achieving a 100% recovery rate across the Monte Carlo study and defining a maximum recoverable tilt of approximately 12° to 15°, the physical boundary of stable flight. The broader aim of this work is to demonstrate that advanced control techniques like thrust vectoring are achievable at the hobby and educational scale, and to provide a foundation that other students can build on.

Keywords: thrust vector control; model rocketry; PID control; Kalman filter; sensor fusion; attitude stabilization; gimbal actuation; flight software; autonomous state machine; Monte Carlo simulation; control authority; inertial measurement unit; finless rocket; aerospace engineering.

1 Introduction

Thrust vector control (TVC) is a fundamental technology in modern rocketry. It enables in-flight trajectory correction and attitude stabilization by redirecting the thrust generated by a solid or liquid powered rocket engine on a 2-axis gimbal. Since its development, TVC has played a crucial role in major space launch vehicles, from the Saturn and Space Shuttle programs through to modern vehicles such as SpaceX's Falcon 9 and NASA's Space Launch System (SLS) (1). TVC allows the vectoring and angling of the thrust-producing component in order to counteract aerodynamic instabilities and initial launch trajectory errors.

While TVC is a core part of large-scale rockets, it remains niche and underdeveloped in model rocketry due to its high complexity and the manufacturing precision required, and most model rockets have relied on traditional, passive, aerodynamic stabilization using fins instead. The most notable exception is the hobbyist work of Joe Barnard's BPS.space project, which has demonstrated thrust-vectored model rockets capable of powered, controlled flight and even propulsive landing (2). This paper builds on that precedent. It sets out the system's control mathematics and flight software in enough detail to serve as an educational and reproducible reference for other students.

The system described here is a compact, low-cost, and fully functional thrust vector control package, integrating a gimbal mount for thrust deflection, a high-speed PID control loop for attitude correction, and a Kalman filter for state estimation during powered flight. The vehicle is designed to operate autonomously, from launch through recovery, transitioning through predefined flight states based on real-time sensor data.

The ultimate objective of this work is to demonstrate that TVC can be successfully adapted to small-scale rocketry, and thereby increase the accessibility of controllable, reusable, and autonomously governed model rockets to students and hobbyists. The paper also provides detailed engineering insight into system design, flight software development, and simulation-based validation.

The paper follows the structure of a standard research report, with Sections 2 through 5 forming the Materials and Methods, Section 6 the Results and Discussion, and Section 7 the Conclusion.

2 System Architecture Overview

This section describes the mechanical, electrical, and computational architecture of the thrust-vector-controlled model rocket. Particular emphasis is placed on component selection, system integration, and the engineering tradeoffs required to achieve reliable closed-loop control within the constraints of a small-scale vehicle.

2.1 Mechanical Airframe & Structural Elements

The rocket airframe was designed around a standard 76 mm (3 in) cardboard body tube manufactured by Estes® Rockets. This diameter provided sufficient internal volume for the thrust vectoring gimbal, avionics bay, and recovery system while maintaining a manageable mass and aerodynamic profile. Custom structural components were designed in Fusion 360 and fabricated using fused

deposition modeling (FDM) 3D printing with PLA filament.

The nose cone is a hollow, aerodynamically contoured structure that reduces drag during ascent and houses the recovery system. Its profile follows a conventional low-drag form, shaped to keep frontal area small while leaving room inside for the parachute and its deployment hardware.

The avionics bay serves as the structural and functional core of the vehicle. It houses the flight computer, inertial measurement unit (IMU), power regulation hardware, and wiring harness. The avionics were enclosed within a protective internal sleeve to shield sensitive electronics from combustion byproducts produced during black powder motor operation, including carbon dioxide, nitrogen, and carbon monoxide. This enclosure also provided structural isolation from exhaust turbulence and vibration.

The thrust assembly consists of the motor mount, retention system, and exhaust shielding designed to interface directly with the TVC gimbal. The motor mount was 3D printed at high infill density to keep it rigid under thrust loading. The added mass of a dense print is a worthwhile tradeoff here, since a stiff mount holds the motor centerline fixed and prevents shifts in the center of mass that would otherwise couple into the control loop. An Estes® E12-4 solid rocket motor was selected for this prototype due to its well-characterized thrust curve and compatibility with the airframe scale. The motor mount incorporates mechanical retention screws positioned beneath the solid propellant grain to prevent axial movement during ignition and powered flight. When the gimbal sits at its neutral position, the motor centerline lines up with the rocket's longitudinal axis. Keeping that alignment matters because any offset between the thrust line and the center of mass would produce a steady turning moment even with no gimbal command, which the controller would then have to cancel continuously.

Passive aerodynamic fins were intentionally omitted from the design. This decision allowed thrust vector control to serve as the sole stabilization mechanism, more closely modeling real-world TVC applications such as the Falcon 9 first stage. While active or passive fins can provide stabilization at higher airspeeds, their exclusion ensured that all attitude correction authority originated from the gimbal system alone.

2.2 Thrust Vectoring (TVC) System Design

The active stabilization system is centered around a two-axis thrust vector control gimbal that allows the rocket motor to deflect independently in pitch and yaw relative to the vehicle's longitudinal axis. This capability enables real-time corrective torque application during powered flight.

The gimbal mechanism (Figure 1) consists of a nested, dual-axis structure mounted within the aft section of the airframe. It was designed as a multi-part PLA assembly to maximize available degrees of freedom while maintaining sufficient stiffness under thrust loading. Two MG90S digital micro servos actuate the gimbal, selected for their favorable torque-to-weight ratio, rapid response time, and ease of integration with the flight computer. The manufacturer datasheet rates the MG90S at 0.10 s per 60° under no-load conditions (3); this paper uses a more conservative figure of 0.12 s per 60° throughout, reflecting the additional load imposed by the gimbal mechanism and motor assembly relative to the unloaded datasheet rating.

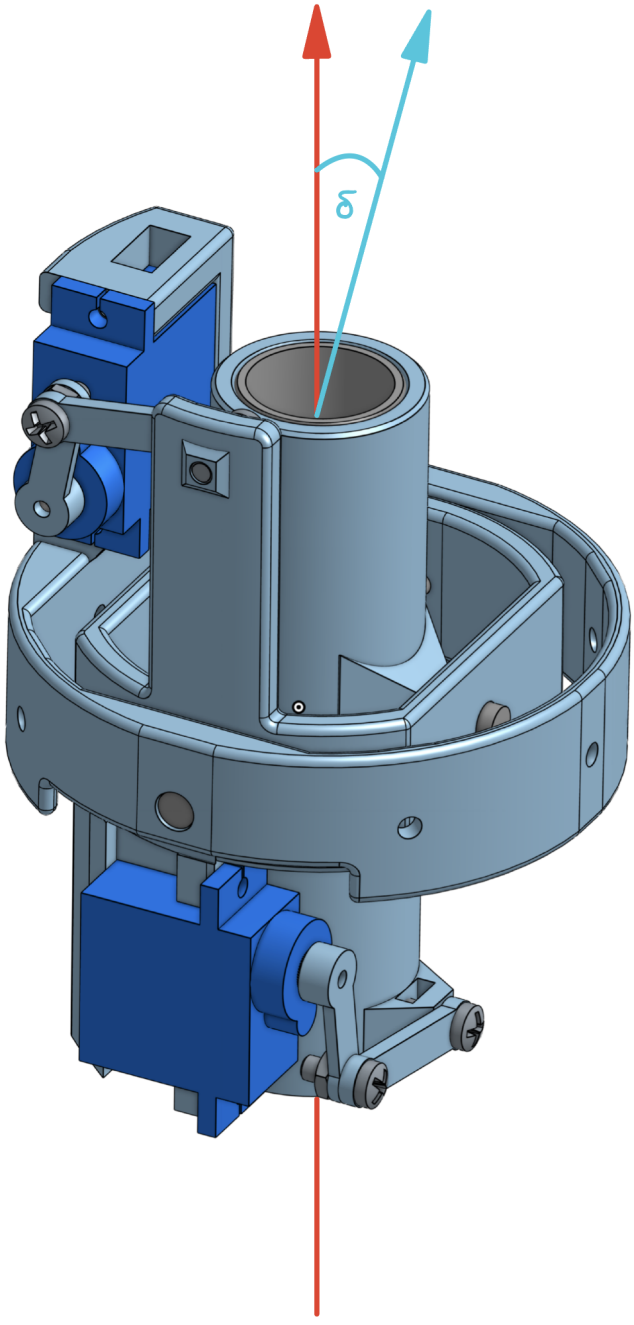

Figure 1: Two-axis thrust vectoring gimbal. The nested structure pivots the motor about independent pitch and yaw axes, each driven by an MG90S servo through a linkage. The deflection angle δ is measured between the motor centerline and the vehicle's longitudinal axis.

The gimbal provides a maximum deflection range of ±10 degrees in both pitch and yaw. This range was determined through simulation and mechanical testing to provide sufficient corrective authority for a rocket of this size while avoiding excessive coupling effects or mechanical interference. Because corrective torque is directly proportional to the deflection angle, this limit defines the upper bound of achievable control authority during powered flight.

2.3 Avionics & Sensor Suite

Vehicle orientation and acceleration are measured using an MPU-6050 inertial measurement unit, which integrates a three-axis gyroscope and three-axis accelerometer into a single module. The gyroscope provides high-rate angular velocity measurements suitable for short-term attitude estimation, while the accelerometer gives an absolute orientation reference whenever the vehicle is not accelerating drastically, since gravity then dominates its reading and points reliably downward. The sensor communicates with the flight computer over I^2C, a standard two-wire serial bus used for short-range links between chips. Although the MPU-6050 lacks an onboard magnetometer or barometric sensor, its high update rate, affordability, and compatibility with embedded development platforms made it suitable for this prototype. The missing magnetometer leaves the roll axis without an absolute heading reference, a limitation examined in Section 3.2. The missing barometer reflects the scope of this hardware revision. The flight computer described here was built for two-degree-of-freedom static testing, where the vehicle stays fixed to the stand and altitude is never measured, so a pressure sensor would serve no purpose. A free-flight version of the vehicle would instead carry a barometer to support apogee detection, as described in Section 4.5. The remaining limitations are handled through the software-based sensor fusion of Section 3.2. Flight data is logged to the Teensy® 4.1's onboard SD card at the loop rate. Logged parameters include IMU measurements, gimbal deflection angles, and state transitions. This resolution supports detailed analysis of control-system performance and disturbance response during testing. Several onboard LEDs provide real-time visual feedback of system status, including power availability, IMU initialization, and flight mode. These indicators proved useful during ground testing and pre-launch diagnostics. Power is supplied by a two-cell lithium polymer battery regulated to 5 V via a buck converter. The system is designed to handle transient current spikes caused by servo actuation, with additional capacitive smoothing included to mitigate voltage sag. Modular JST and Dupont connectors were used throughout the wiring harness to facilitate rapid component replacement and reduce the risk of permanent damage during testing.

2.4 Onboard Flight Computer & Software Stack

The flight computer is built around a Teensy® 4.1 microcontroller (10), selected for its high computational throughput, reliability, and integrated storage capabilities. Its ARM Cortex-M7 processor operates at 600 MHz, providing ample processing headroom for real-time control loops, Kalman filtering, and state machine logic without the loop ever running long enough to miss its scheduled update. The board's extensive I/O support allows simultaneous interfacing with the IMU, servos, status indicators, and safety switches.

At the highest level, the system transitions between predefined operational modes including pad

idle, powered flight, apogee detection, descent, recovery, and post-landing (detailed in Section 4 and Appendix A.1).

The flight software is implemented using Arduino-based C++, leveraging standard libraries for I^2C communication, SD card access, and servo control, alongside a custom Kalman filter implementation. This development approach combines the accessibility of the Arduino ecosystem with the computational performance required for advanced control and data analysis.

2.5 Justification for TVC vs. Passive Fins

Compared to passive aerodynamic stabilization methods, thrust vector control offers significantly greater authority during low-speed and early ascent phases, where aerodynamic forces are minimal. Static fins rely on airflow to generate stabilizing moments and therefore become effective only after sufficient velocity has been achieved. In contrast, thrust vectoring allows corrective forces to be applied immediately after ignition, when disturbances and asymmetries are most likely to grow rapidly.

This capability is particularly important for low-mass rockets, which are highly sensitive to wind, thrust misalignment, and manufacturing tolerances. Small deviations in attitude can quickly compound into large trajectory errors if not corrected early. By actively redirecting thrust, the TVC system provides precise and anticipatory corrections, reducing reliance on passive aerodynamic damping. This authority is not without limits. Because it comes entirely from the motor, the gimbal can correct only while the motor is burning, and the torque available rises and falls with the thrust curve, so the system has its greatest authority early in the burn and none at all once the motor stops.

Removing the fins also makes the vehicle aerodynamically unstable, since the center of pressure (CoP) sits ahead of the center of mass (CoM) throughout powered flight. The airframe behaves like an inverted pendulum, the same problem faced by orbital-class boosters, and it stays upright only through constant high-frequency correction. The control loop described in Section 3.1 supplies that correction, adjusting the gimbal far faster than the instability can grow.

2.6 System Integration Considerations and Challenges

Integrating mechanical, electrical, and computational subsystems into a cohesive thrust-vector-controlled vehicle presents significant engineering challenges. The reliability of the rocket depends not only on the performance of individual components but also on their interaction under conditions of high acceleration, vibration, and thermal stress.

Thermal protection mattered because the avionics sit close to the motor. Hot exhaust gas and combustion particulate can damage the electronics, so the motor was positioned to direct its plume away from sensitive structures, and a light PLA shield wrapped in aluminum foil was added to isolate the avionics from the heat.

Vibration was a second challenge, mainly for the inertial sensors, since high-frequency oscillation feeds noise into the IMU and degrades the attitude estimate. To limit it, the IMU was mounted on rubber isolators, and the gimbal mount and avionics bay were cross-braced to resist torsional

flexing. The wiring harnesses were held in place with adhesive and strain relief so they could not shift during launch.

Maintaining alignment between the center of mass (CoM) and center of thrust (CoT) is critical in thrust-vector-controlled systems. Even a small offset between the two creates a coupling effect, in which the thrust line produces an unintended moment that the controller then has to spend authority correcting. The CoM was determined experimentally using a balance method with the fully assembled rocket, including the motor. This measurement was supplemented with a method-of-moments calculation based on individual component masses and positions to increase confidence in the result. The equation for calculating the moments was derived from the standard center-of-mass relation:

$$x_{CoM} = \frac{\sum_i x_i m_i}{\sum_i m_i} \tag{eq1}$$

Because weight is simply mass scaled by the local gravitational acceleration g (i.e., $W_i = m_i g$), the same relation can equivalently be expressed in terms of measured component weights, with g cancelling from numerator and denominator:

$$x_{CoM} = \frac{\sum_i x_i W_i}{\sum_i W_i} \tag{eq2}$$

where W_i is the i-th component's weight and x_i is its distance from the fixed reference point. This equivalence allowed the balance-test measurement, which directly measures weight, to be cross-checked against the method-of-moments calculation based on component masses, increasing confidence in the result.

The center of pressure (CoP) was estimated using OpenRocket (4) simulation software, which applies the Barrowman equations (5) to model aerodynamic forces on slender bodies. Center of pressure is governed by the equation:

$$x_{CoP} = \frac{\sum_i C_{N_i} x_i}{\sum_i C_{N_i}} \tag{eq3}$$

where C_{N_i} is the normal force coefficient for each section, and x_i represents its axial distance from the nose tip. The final configuration showed the CoP positioned approximately 80 mm ahead of the CoM, consistent with the divergently unstable, finless configuration described in Section 2.5: with no aerodynamic surfaces to generate a restoring moment, the vehicle's aerodynamic moment grows with tilt rather than opposing it. OpenRocket's output provided both a numerical CoP location and an estimated margin given by:

$$\text{Static Margin} = \frac{x_{CoP} - x_{CoM}}{D} \tag{eq4}$$

where D is the rocket body diameter (76 mm in this design) and x is measured from the nose tip, increasing toward the tail. With the CoP ahead of (i.e., at a smaller x than) the CoM, this expression yields a negative static margin. Static margin measures the gap between the center of pressure and the center of mass in body diameters, and its sign decides stability. A positive margin, with the CoP behind the CoM, gives a self-correcting airframe, while the negative value here means any tilt grows rather than corrects, which is the expected and intended signature of a finless vehicle.

Propellant burn during powered flight causes the CoM to shift forward as mass is consumed from the aft motor, narrowing the CoP–CoM gap and modestly reducing the magnitude of the instability over the course of the burn, though the configuration remains aerodynamically unstable throughout powered flight. Because the airframe never becomes self-correcting, holding it stable depends entirely on the control system rather than on aerodynamics. The vehicle stays upright only for as long as the gimbal can supply more corrective torque than the aerodynamic moment demands, and Section 3.4 quantifies the tilt angle beyond which it no longer can.

With the structural, aerodynamic, and electronic foundations established, the next phase of the project was centered on developing the closed-loop control system responsible for maintaining rocket stability during powered flight. These physical and simulated findings directly informed the PID tuning process and the Kalman filter parameters, ensuring that the control inputs were responsive and accurate during rapidly changing acceleration and vibrations. The following section details the mathematical framework, algorithmic implementation, and practical tuning procedures that governed the TVC's behavior in real time.

3 Control Theory and PID Stabilization

TVC requires real-time orientation correction to counteract disturbances such as wind, manufacturing tolerances, or uneven thrust. This is achieved through a closed-loop control system that combines an IMU, a Kalman filter, and a dual-axis PID controller, continuously adjusting the angle of the motor to correct pitch and yaw deviations.

3.1 Orientation Control using PID

At the core of the TVC system is a Proportional–Integral–Derivative (PID) controller, which stabilizes the rocket by adjusting the motor's deflection angle in real time. The controller opposes any external force that pushes the rocket away from a vertical attitude (0° tilt), keeping the thrust aligned with the rocket's center of mass. It does this by minimizing the angular error between the desired vertical orientation and the rocket's measured tilt, using attitude data derived from the IMU. Pitch and yaw are stabilized independently: each axis is treated as a separate control loop, with its own error signal, feedback, and correction command sent to the servo motors.

The PID control law is:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d\, \frac{de(t)}{dt} \tag{eq5}$$

where:

- $u(t)$: the control output, i.e. the commanded servo deflection angle.
- $e(t)$: the angular error at time t, defined as the difference between the measured attitude and the nominal vertical orientation of 0°.
- $K_p,\ K_i,\ K_d$: the tuned proportional, integral, and derivative gains, respectively.

Each term performs a distinct role. The proportional term (K_p) applies a correction in direct proportion to the current angular error, providing the immediate response to a deviation; if it is set too high, however, it can drive the system into oscillation. The integral term (K_i) accumulates the error over time and removes any steady-state offset, ensuring the rocket returns to exactly vertical rather than settling at a small persistent tilt; an overly large integral gain can cause overshoot or slow oscillatory drift. The derivative term (K_d) responds to the rate at which the error is changing, anticipating where the error is heading and damping the response before it overshoots. This improves stability, but because it acts on the rate of change of a noisy signal, an excessive derivative gain amplifies measurement noise, which is a particular concern given that IMU data is inherently noisy. This sensitivity is one of the main reasons the loop relies on the Kalman filter of Section 3.2, which smooths the attitude estimate before the derivative term differentiates it. For the validation simulations presented in Section 6, the integral term was set to zero ($K_i = 0$). Over a burn lasting only a few seconds, the steady-state-error correction the integral term provides is of limited benefit, while the windup risk described above is a genuine concern for a fast, short-duration burn; the simulated controller therefore operates as a pure PD loop. The full PID control law, including integral action and the windup protection described in Appendix A.3, remains implemented in the flight software for longer powered burns, where a sustained thrust phase makes steady-state offset correction worthwhile. Tuning these three gains requires iterative ground and simulated testing to reach stability without excessive oscillation. Initial values can be estimated using the Ziegler–Nichols heuristic method (6). The method works by raising the proportional gain alone until the system holds a steady oscillation, then reading off that critical gain and the oscillation period and using them to set the proportional, integral, and derivative terms from standard formulas. It is deliberately aggressive and tends toward high gains, which does not suit a vehicle that needs stability and minimal overshoot. These initial values were therefore treated only as a starting point and refined empirically through static-fire testing, confirming that servo commands stayed within their actuation limits and that oscillation was minimized during rapid attitude correction.

The PID model has an important limitation: it assumes a linear relationship between the input (servo commands) and the output (angular deflection). During powered flight this relationship is in fact nonlinear, owing to aerodynamic effects, a center of mass that shifts as propellant burns, and particularly the time-varying thrust curve of a solid rocket motor. To keep these nonlinearities from destabilizing the loop, the derivative term can be tuned conservatively, and integral windup protection can be applied so that error does not continue to accumulate once the gimbal has reached its maximum deflection.

3.2 Sensor Fusion using a Kalman Filter

The control loop needs an accurate, real-time estimate of the rocket's orientation to act on, and neither sensor on the MPU-6050 can supply one alone. The gyroscope measures angular velocity accurately over short intervals, but recovering orientation from it requires integration, so its small errors build up and the estimate drifts. The accelerometer measures the direction of gravity and gives an absolute tilt reference, though under powered flight the motor's vibration and acceleration corrupt its readings. Each sensor is strong where the other is weak, the gyroscope over short spans and the accelerometer over long ones. A Kalman filter (7) fuses the two into one orientation

estimate that is more trustworthy than either on its own.

Before stating the filter equations, it is worth fixing the notation, since the subscripts carry specific meaning. The integer k labels the current control cycle, and $k-1$ the previous one. A caret denotes an *estimate* rather than a measured or true value, so $\hat{x}$ is the filter's estimate of the tilt angle. A subscript written as $k|k-1$ is read as "the value at step k given only the information available through step $k-1$": it is therefore a prediction, formed before the new measurement at step k is taken into account. Finally, because each axis (pitch and yaw) is estimated independently as a single tilt angle, every quantity below is a single number rather than a matrix, which simplifies the standard Kalman expressions.

Each cycle proceeds in two steps. In the **prediction** step, the filter advances its previous orientation estimate using the angular velocity reported by the gyroscope:

$$\hat{x}_{k|k-1} = \hat{x}_{k-1} + \omega_k \, \Delta t \tag{eq6}$$

where $\hat{x}_{k-1}$ is the orientation estimate carried over from the previous cycle, ω_k is the angular velocity measured by the gyroscope during the current cycle, Δt is the time elapsed since the previous cycle, and $\hat{x}_{k|k-1}$ is the resulting predicted orientation, before any correction. Put simply, the new angle is the previous angle plus the amount the rocket is estimated to have rotated since the last update.

In the **update** step, the filter corrects this prediction using the accelerometer, weighting the correction by how much it currently trusts that reading:

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k \left(z_k - H\hat{x}_{k|k-1} \right) \tag{eq7}$$

where z_k is the tilt angle derived from the accelerometer during the current cycle, $\hat{x}_k$ is the final, corrected orientation estimate for the cycle, K_k is the Kalman gain (defined below), and H is the observation model, which relates the estimated state to the measured quantity. Because the filter estimates exactly the tilt angle that the accelerometer reports, H equals 1 in this implementation and can effectively be ignored. The term in parentheses, $z_k - H\hat{x}_{k|k-1}$, is the difference between what the accelerometer measures and what the prediction expected, and it serves as the filter's error signal for the current cycle.

The Kalman gain K_k determines how strongly that correction is applied: a large gain pulls the estimate toward the accelerometer, while a small gain leaves it close to the gyroscope-based prediction. The gain is not fixed but is recomputed every cycle from the filter's own estimate of its uncertainty:

$$K_k = \frac{P_{k|k-1} H^T}{H P_{k|k-1} H^T + R} \tag{eq8}$$

Here $P_{k|k-1}$ is the *predicted error covariance* - the filter's uncertainty about its own prediction -

and R is the *measurement noise covariance*, a fixed value representing how noisy the accelerometer is expected to be. Both are variances: single numbers, in units of angle squared, where a larger value simply means greater uncertainty. The superscript T denotes the matrix transpose used in the general formulation; since $H = 1$ here, H and H^T drop out and the gain reduces to the more transparent form

$$K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R} \tag{eq9}$$

In this form the behavior of the gain is clear. The predicted uncertainty $P_{k|k-1}$ and the accelerometer noise R compete in the denominator, and the gain always falls between 0 and 1. When the accelerometer is reliable relative to the prediction, R is small, the gain approaches 1, and the measurement is trusted heavily; when vibration makes the accelerometer unreliable, R is effectively large, the gain approaches 0, and the filter relies on the gyroscope-based prediction instead.

The gain can adapt in this way only because the uncertainty P is itself updated every cycle, in two steps that mirror the orientation update. During prediction, integrating the gyroscope adds uncertainty, so P grows by the process noise Q:

$$P_{k|k-1} = P_{k-1} + Q \tag{eq10}$$

where P_{k-1} is the uncertainty carried over from the previous cycle and Q is a fixed value representing the error the gyroscope introduces each step. After the accelerometer correction is applied, the uncertainty shrinks:

$$P_k = (1 - K_k H)\, P_{k|k-1} \tag{eq11}$$

where P_k is the updated uncertainty passed forward to the next cycle (again with $H = 1$, so the factor is simply $1 - K_k$). Taken together, the two orientation equations and these two covariance equations form a single self-correcting cycle: at every step the filter predicts the new orientation and its uncertainty, measures the disagreement with the accelerometer, and corrects both.

The two fixed quantities Q and R are what the filter is tuned on, and only their relative size matters. A larger Q tells the filter that its gyroscope-based prediction is less trustworthy, which raises the gain and pulls the estimate toward the accelerometer; a larger R does the opposite, favoring the smoother prediction. For this vehicle, the two were chosen so that the estimate stayed responsive during rapid attitude changes without amplifying the accelerometer's vibration noise.

The result is a smooth orientation estimate that suppresses both the short-term vibration noise of the accelerometer and the long-term drift of the gyroscope, and it is this estimate that the PID controller acts on. One limitation follows directly from the sensor suite: the accelerometer can only sense tilt away from vertical, which fixes the pitch and yaw axes, but it carries no information about rotation about the rocket's longitudinal axis. Because the MPU-6050 has no magnetometer to supply an absolute heading reference (8), that rotation (roll) is tracked by the gyroscope alone

and slowly drifts, since integrating the gyroscope's small measurement errors over time lets them accumulate with nothing to correct them. For the short, vertical, and approximately symmetric flights considered here, uncorrected roll does not affect the rocket's ability to maintain a vertical trajectory, so a magnetometer was not included in the design.

In simulation, using the MPU-6050's datasheet noise characteristics (Section 6.7), the filter cut instantaneous tilt noise by roughly 70 to 80%. This reduction matters most for the derivative term, which acts on the rate of change of the estimate and would otherwise amplify raw sensor noise into the gimbal command, so a cleaner estimate allows a more responsive loop without that penalty. The figure is a simulation result based on datasheet noise, and the real motor environment may be harsher, so further gains would likely come from better mechanical isolation of the IMU or a lower-noise sensor rather than from the filter alone.

3.3 Stability Constraints and PID Gain Tuning Methodology

Designing a closed-loop control system for a thrust-vector-controlled rocket required careful attention to both control authority and physical response constraints. The controller's ability to stabilize the vehicle depended not only on well-tuned gains but also on the mechanical, aerodynamic, and computational limits of the components selected for this project.

The most significant of these constraints was the servo response speed relative to the rate at which the rocket's instability grows. The MG90S servos used in the gimbal have a rated actuation speed of roughly 0.12 s per 60° of travel (3). A full deflection therefore takes roughly a tenth of a second, which corresponds to an effective control bandwidth of about 8–10 Hz (since $1/0.12\ \text{s} \approx 8$ actuations per second). The control loop on the Teensy® 4.1 microcontroller was run at 150 Hz, far above this actuator bandwidth, so that the controller could detect and begin responding to a deviation well before the servo itself became the limiting factor, preventing small errors from propagating into larger oscillations.

The control authority of the system, defined as the total corrective torque available from gimbal deflection, is set by the motor thrust and the moment arm between the thrust line and the center of mass. The corrective moment is:

$$\tau = F_T \cdot r \cdot \sin\delta \tag{eq12}$$

where F_T is the instantaneous motor thrust, r is the distance from the center of mass to the gimbal pivot, and δ is the gimbal deflection angle. Because torque scales with $\sin\delta$ and the gimbal's range is limited to $\pm 10°$, the maximum corrective torque is bounded by that mechanical range. The PID deflection commands were therefore constrained to the same $\pm 10°$ window, since the controller cannot be allowed to request a correction the gimbal cannot physically produce. Commanding more deflection than the hardware can deliver is known as actuator saturation, and limiting the control output prevents it. At the motor's peak thrust of 33.0 N (Section 6.2), this corresponds to the largest restoring moment the vehicle can generate, a figure quantified across the full burn in Section 6.8.

3.4 Limits of Control Authority and Tilt Recovery

Whether a thrust-vector-controlled rocket can hold or recover stable flight comes down to its control authority, meaning the corrective torque the gimbal can supply measured against how fast the angular disturbance is growing. Once a disturbance outpaces what the gimbal can counter, the rocket can no longer return to vertical and the flight is lost. Several factors set this authority together. Servo speed and motor thrust fix how much corrective torque is available, while aerodynamic forces, structural geometry, and sensor latency each limit how effectively that torque reaches the vehicle. The subsections that follow examine these limits and establish the theoretical maximum recoverable tilt angle.

The rotational motion of the rocket about its center of mass is governed by the rotational form of Newton's second law, the angular analog of $F = ma$,

$$I\,\ddot{\theta} = \tau_{\text{net}} \tag{eq13}$$

where I is the rocket's moment of inertia about the pitch (or yaw) axis, θ is the tilt angle from vertical, $\ddot{\theta}$ is the resulting angular acceleration, and τ_{net} is the sum of all torques acting on the vehicle. That net torque is the difference between the corrective torque supplied by the gimbal and the disturbance torque that drives the rocket away from vertical:

$$I\,\ddot{\theta} = \tau_{\text{control}} - \tau_{\text{dist}}(\theta) \tag{eq14}$$

Recovering or holding a stable attitude means keeping $\ddot{\theta}$ directed back toward vertical, which requires the corrective torque to match or exceed the disturbance across the range of tilts the rocket actually reaches. This relationship is the foundation of the analysis that follows: the corrective torque is bounded above by the gimbal's mechanical limit (Section 3.4.1), the disturbance torque grows with tilt (Section 3.4.2), and the angle at which the two become equal marks the edge of recoverable flight.

3.4.1 Corrective Torque Limitations

The total corrective torque generated by the TVC system is set by the motor thrust, the offset between the gimbal and the center of mass, and the gimbal's maximum deflection angle:

$$\tau_{\text{max}} = F_T \cdot r \cdot \sin(\delta_{\text{max}}) \tag{eq15}$$

where:

- F_T is the motor thrust.
- r is the distance from the center of mass to the gimbal pivot.
- δ_{max} is the maximum gimbal deflection angle.

This equation captures the first hard limit of the system. Because δ_{max} is fixed by the gimbal's

mechanical range, the corrective torque the vehicle can generate at a given thrust is also fixed. As the rocket tilts farther from vertical, it requires progressively more torque to counteract the gravitational and aerodynamic moments acting on it. Once the disturbance torque exceeds $\tau_{\max}$, recovery becomes impossible, no matter how precisely the PID controller is tuned.

3.4.2 Maximum Recoverable Tilt Angle

The maximum recoverable tilt angle is the largest angular deviation from vertical from which the TVC system can still return the rocket to a stable orientation during powered flight. It is found by comparing the maximum corrective torque available from thrust vectoring against the combined disturbance torques acting on the vehicle.

The dominant disturbance for this analysis is the destabilizing aerodynamic moment, which grows with angle of attack and airspeed. Two terms that might appear to belong here do not. Gravity acts through the center of mass and so produces no moment about that point, and the $I\ddot{\theta}$ term in the rotational equation of motion is the vehicle's own inertial response to the net torque rather than a disturbance in its own right.

At small tilt angles, the aerodynamic and gravitational torques scale approximately linearly with the tilt. As the tilt increases, however, these moments grow faster than the available corrective torque, which is capped by the fixed gimbal limit. Once the disturbance torque exceeds the maximum corrective torque defined in Section 3.4.1, recovery is no longer possible even with thrust still available. This defines a recovery boundary:

$$\tau_{\text{dist}}(\theta) \leq \tau_{\max} \tag{eq16}$$

where θ is the rocket's tilt angle from vertical, distinct from the gimbal deflection δ defined earlier. Beyond this boundary the system diverges: further corrective commands can no longer generate enough angular acceleration to reverse the motion. In practical terms, there is a finite tilt angle within which the rocket can still be brought back to vertical, and past it the flight becomes unrecoverable. To make this boundary concrete, the disturbance torque must be written in terms of the tilt angle. For the finless configuration the dominant disturbance is aerodynamic: with the center of pressure ahead of the center of mass, any tilt produces a destabilizing moment about the center of mass. For a near-vertical trajectory the tilt angle and the aerodynamic angle of attack are approximately equal, and at small angles the aerodynamic normal force is

$$N \approx \tfrac{1}{2}\rho v^2 A_{\text{ref}} C_{N\alpha}\, \theta \tag{eq17}$$

where ρ is the air density, v is the airspeed, A_{ref} is the reference cross-sectional area, and $C_{N\alpha}$ is the normal-force-coefficient slope, which sets how steeply the aerodynamic side force grows as the angle of attack increases, obtained here from the Barrowman method (5). This force acts at the center of pressure, a distance $\ell = x_{\text{CoP}} - x_{\text{CoM}}$ from the center of mass, so the disturbance torque is

$$\tau_{\text{dist}}(\theta) \approx \tfrac{1}{2}\rho v^2 A_{\text{ref}} C_{N\alpha}\, \ell\, \theta \tag{eq18}$$

Setting this equal to the maximum corrective torque $\tau_{\max} = F_T\, r \sin(\delta_{\max})$ from Section 3.4.1 and solving for the tilt gives the maximum recoverable angle:

$$\theta_{\max} \approx \frac{F_T\, r \sin(\delta_{\max})}{\tfrac{1}{2}\rho v^2 A_{\text{ref}} C_{N\alpha}\, \ell} \tag{eq19}$$

This form makes the dependence clear: the recoverable tilt rises with motor thrust F_T and falls as airspeed increases, since the aerodynamic disturbance scales with v^2. The rocket is therefore easiest to recover early in the burn, when thrust is high and airspeed is still low, and progressively harder to recover as thrust decays and dynamic pressure builds.

Evaluated with this vehicle's parameters, the boundary is consistent with the results of simulation and analysis, which indicated that stable recovery was consistently achievable for pitch and yaw deviations below approximately 12° and matched the 12°–15° envelope mapped numerically in Section 6.10. This value is slightly larger than the mechanical gimbal limit of ±10° because recovery depends on angular acceleration applied over time, not on deflection angle alone.

3.4.3 Dynamic Effects and Control Latency

Beyond the static torque limit, tilt recovery is also constrained by dynamic effects: control latency, actuator response speed, and sensor update rate. Even when sufficient corrective torque exists in principle, delays in any of these can allow an angular deviation to grow past the recoverable limit before the correction is fully applied. In an ideal system this delay would be zero, but in practice even a few milliseconds of lag can cause the controller to overcorrect or undercorrect.

The MG90S servos have a finite response time of roughly 0.12 s per 60° of motion, which limits how quickly the thrust vector can be redirected against a sudden disturbance.If the rocket tips off rapidly right after launch, which lightweight vehicles are especially prone to, the servo can lag the disturbance and let angular velocity build faster than the system can arrest it. Sensor processing adds a smaller delay of its own: although the IMU reports data frequently, the Kalman filter smooths rapid jumps in that data to reject noise, which introduces a small but unavoidable phase lag in the attitude estimate. This trade-off improves stability under vibration at the cost of some responsiveness to very fast disturbances.

Together, these effects impose a dynamic recovery limit, one where recovery fails not because the available torque is insufficient, but because the correction arrives too late. Two design choices reduce this risk. Conservative PID gains keep the response stable and predictable rather than aggressive, and running the control loop well above the actuator bandwidth keeps computational delay small so that corrections go out as early as possible. Taken together, the static and dynamic limits define a specific, calculable domain of controllability within which stable flight and recovery are possible. Characterizing this domain was essential both to the mechanical design of the gimbal and to the tuning of the closed-loop control algorithm.

4 Flight Software and Autonomous State Transitions

The flight software serves as the layer that governs when the control system is enabled, how sensor data is interpreted, and how the system transitions between distinct phases of flight. Although the thrust vector control system defines how corrective actions are computed, the flight software determines when such corrections are physically necessary and safe to apply. For this project, an autonomous state-machine-based architecture was implemented to ensure data-backed behavior, robustness that protects against sensor noise, and safe transitions between powered and unpowered flight. The software executes on a Teensy® 4.1 microcontroller as a high-frequency loop (~200 Hz). Within this loop, the PD/PID computation and Kalman filter update specifically execute at 150 Hz, matching the control update rate used in the validation simulations of Section 6, while sensor sampling, state-machine evaluation, and data logging proceed at the full loop rate.

4.1 Autonomous Staging and Flight Phase Identification

At the core of the flight software lies the autonomous staging system, responsible for identifying the rocket's current phase of flight based on real-time sensor data. This system reads the accelerometer and gyroscope, along with quantities derived from them such as integrated velocity and tilt angle, to decide whether the rocket is idle, under powered ascent, coasting, descending, or has landed. Each phase carries its own control context, so the gimbal corrects only while thrust is available and the recovery logic arms only when the system deems it necessary. Rather than relying on a single sensor or threshold, the staging logic evaluates multiple indicators simultaneously. This multi-parameter approach improves robustness against noise, vibration, and transient disturbances, particularly during high-dynamic events such as launch and motor burnout. The overall staging logic follows the structure illustrated in Listing 1, which outlines the transitions between flight states.

4.2 State Machine Logic and Execution Flow

The flight software is structured as a state machine operating within a continuous control loop. During each iteration of the loop, current sensor measurements are processed and compared against state-dependent transition criteria. The system begins in a pad idle state, during which the rocket is stationary and awaiting launch. In this state, sensor calibration is completed, baseline offsets are established, and all actuators are held in neutral positions to prevent unintended motion.

Upon detection of launch conditions, the system transitions into the powered flight state. In this phase, thrust vector control is fully enabled, and the PD controller actively stabilizes pitch and yaw using Kalman-filtered attitude estimates. The powered flight state persists until motor burnout and upward velocity decay are detected. The phases that follow belong to the intended free-flight design rather than the ground-constrained system validated here (Section 4.5). In free flight, the software would transition into an apogee detection state, distinguishing between transient velocity fluctuations and the true apex of the trajectory. Once apogee was confirmed, the system would enter the descent state, where active thrust vector control is disabled, since no thrust remains to generate corrective torque. A final transition would follow once landing was detected, leaving the system in

a recovery or post-landing state where data logging is finalized and active control subsystems are powered down.

This state-based execution model ensures that control logic is always context-aware and prevents undesired actuation when it is not required or will not have a meaningful effect on controlling the rocket's trajectory.

4.3 Sensor Inputs, Calibration, and Filtering

Reliable autonomous decision-making depends on accurate and stable sensor data. The rocket employs a six-degree-of-freedom MPU-6050 inertial measurement unit, providing three-axis accelerometer and gyroscope measurements. While these sensors offer high sampling rates and low latency, their raw outputs are susceptible to bias, drift, and vibration-induced noise. Gyroscopes measure angular velocity and therefore accumulate integration error over time, leading to gradual drift in estimated orientation. Accelerometers, while capable of providing absolute tilt references, are sensitive to high-frequency vibration and non-gravitational accelerations during powered flight. To address these limitations, sensor calibration routines can be performed prior to launch to remove static offsets and real-time filtering is applied during flight. Filtered sensor data is then passed to the Kalman filter described in Section 3.2, producing a smooth and reliable estimate of the rocket's orientation. This filtered attitude estimate is used both for control input during powered flight and for flight state determination throughout the mission.

4.4 Launch Detection Criteria

Accurate launch detection is critical for enabling thrust vector control at the correct moment. Premature activation can lead to unnecessary servo actuation on the launch rail, while delayed activation reduces the system's ability to counter early disturbances. To balance responsiveness and robustness, launch detection is based on multiple simultaneous conditions rather than a single acceleration threshold.

The transition from pad idle to powered flight is triggered only when sustained axial acceleration exceeding a predefined threshold is detected, accompanied by a consistent increase in vertical velocity over a short time window. Additionally, a minimum arming time must elapse to prevent accidental activation during handling, and gyroscope noise must remain below a defined threshold to ensure a stable sensor environment. Together, these conditions significantly reduce the probability of false positives while ensuring that thrust vector control engages directly after liftoff.

4.5 Apogee Detection Algorithm

The remaining flight phases, beginning with apogee detection, belong to the free-flight vehicle rather than to the ground-constrained system validated in this paper. They are presented here as the intended architecture, since the test-stand rocket is fixed in place and never ascends or descends. In free flight, apogee marks the critical handoff from ascent to descent logic. A thrust-vectored rocket can show oscillatory velocity behavior near the peak, which makes a single zero-crossing

of vertical velocity an unreliable trigger, so the intended design combines several conditions rather than relying on one.

In that design, apogee would be confirmed only when several indicators agree. Negative vertical velocity must persist over a defined interval, and barometric altitude, supplied by the pressure sensor the free-flight vehicle would carry, must stop increasing. A minimum time since launch guards against false detections during early flight transients, and pitch and yaw must stay within acceptable bounds so that an unstable attitude does not trigger a premature transition.

Once these conditions were met, the system would transition into the descent phase and begin recovery sequencing. A timeout safeguard would force the transition if apogee were not detected within a predefined window, so that recovery logic could never be delayed indefinitely, and each apogee event would be logged for later review.

4.6 Descent and Recovery Phase

In the free-flight design, the rocket would enter the descent phase following apogee. Thrust vector control is disabled here and the gimbal mechanically locked, since no thrust remains to generate corrective torque. Sensor monitoring would continue, tracking barometric altitude and vertical velocity for landing detection and later analysis.

Recovery mode would begin once the vehicle descended below a safe altitude and its vertical velocity stayed low for a sustained period, indicating ground contact. Final data would then be written to storage and power to the active control components shut down, leaving the system in a passive idle state awaiting retrieval.

This approach prevents post-landing data corruption and protects actuators from damage due to unnecessary motion after touchdown.

4.7 Abort Mechanisms and Fail-Safe Logic

To ensure system safety and data integrity, multiple fail-safe mechanisms are embedded within the flight software. These safeguards monitor for abnormal servo behavior, sensor faults, and missed state transitions. In the event of excessive servo command rates, thrust vector control is immediately disabled and the fault is logged. If apogee is not detected within the expected time window, recovery mode is forced to prevent indefinite powered logic execution.

Manual abort capability is also provided during pre-launch conditions, allowing the system to return safely to a standby state in the event of sensor anomalies or excessive pre-launch tilt. In the case of system crashes or unexpected resets, periodic non-volatile memory backups ensure that partial flight data can still be recovered.

4.8 Future Enhancements to Flight Logic

While the current flight software architecture is able to demonstrate reliable performance, several enhancements could further improve robustness and accuracy. Integrating a high-resolution baro-

metric pressure sensor would provide an independent altitude measurement for improved apogee detection, though care must be taken to mitigate pressure disturbances caused by airflow. Adaptive control strategies, such as in-flight PID gain adjustment or model predictive control, could further optimize stabilization performance under varying flight conditions. Additionally, extending the state machine to support multi-stage propulsion or stage separation detection would enable more complex flight sequences.

5 Simulations and Performance Modeling

Simulation and modeling played a central role in validating the feasibility of the thrust vector control system prior to experimental flight testing. Given the risks associated with uncontrolled instability in TVC rockets, analytical and numerical models were used to predict vehicle behavior under both ideal and disturbed conditions, establishing performance expectations against which experimental data could later be compared. Rather than rely on a single framework, the project used two complementary models. OpenRocket (4) handled baseline trajectory behavior and aerodynamic stability through the established Barrowman method (5), and a set of custom analytical models covered thrust vector control authority, angular response, and the recovery limits. This separation allowed aerodynamic and control effects to be studied independently before being treated as a coupled system. The OpenRocket baseline model reproduced the geometry, mass distribution, and Estes® E12-4 thrust curve of Section 2 and confirmed the static instability anticipated in Section 2.5: the center of pressure remained ahead of the center of mass, reinforcing the necessity of active stabilization.

Two findings from this stage shaped the design directly, and both are quantified later in the validation results of Section 6. The first concerns the timing of control authority. The corrective-torque relationship of Sections 3.3 and 3.4.1 shows that available authority is greatest just after ignition and falls steadily toward burnout, which is why the flight logic of Section 4 concentrates its effort in the early powered phase, when thrust and corrective torque are both at their highest. The second concerns recovery. A rigid-body tip-off model showed that small initial tilts are arrested within a fraction of a second, while tilts beyond a critical angle diverge instead. That result matches the recovery limits derived in Section 3.4 and explains the conservative launch thresholds built into the flight software.

Ultimately, these simulations were not intended to predict flight behavior exactly, but to define a realistic operating envelope for the thrust vector control system. By identifying control authority limits, disturbance sensitivity, and recovery boundaries in advance, they reduced the risk of catastrophic failure during experimental testing.

6 Simulation-Based Validation

Due to airspace constraints, no powered flight testing was conducted. System behavior was evaluated using a physics-based simulation incorporating rigid-body rotational dynamics, thrust vector geometry, actuator limits, and PID control laws.

6.1 Overview of Simulation Objectives

To evaluate the performance and robustness of the thrust vector control system in the absence of powered flight testing, a comprehensive set of physics-based simulations was developed in Python. The simulation integrates rigid-body rotational dynamics using a fourth-order Runge-Kutta (RK4) scheme at a 1 ms time step, with the Estes® E12-4 thrust curve represented by NAR-certified data points interpolated via numpy. The control architecture, which includes the discrete-time PD controller (the integral term is disabled for this simulation, $K_i = 0$, per Section 3.1), Kalman filter, and servo rate limiting, runs at 150 Hz within the simulation loop to match the intended flight computer update rate. It is important to note that the simulations assume a rigid airframe, constant atmospheric conditions, and no launch rail dynamics. These simplifications are discussed further in Section 6.12.

6.2 Motor Thrust Profile

Before examining control system behavior, it is useful to understand the propulsive input driving the simulation. Figure 2 shows the NAR-certified thrust curve for the Estes® E12-4 motor, interpolated from published data points sourced from ThrustCurve.org (9). The motor produces a sharp peak of 33.0 N at approximately $t = 0.29$ s, followed by a gradual decay to a plateau of roughly 9.5 N before burnout at 2.44 s, with a mean thrust of 11.12 N. This time-varying thrust profile is significant for control system design: available corrective torque is directly proportional to instantaneous thrust, meaning control authority is highest immediately after ignition and diminishes steadily toward burnout.

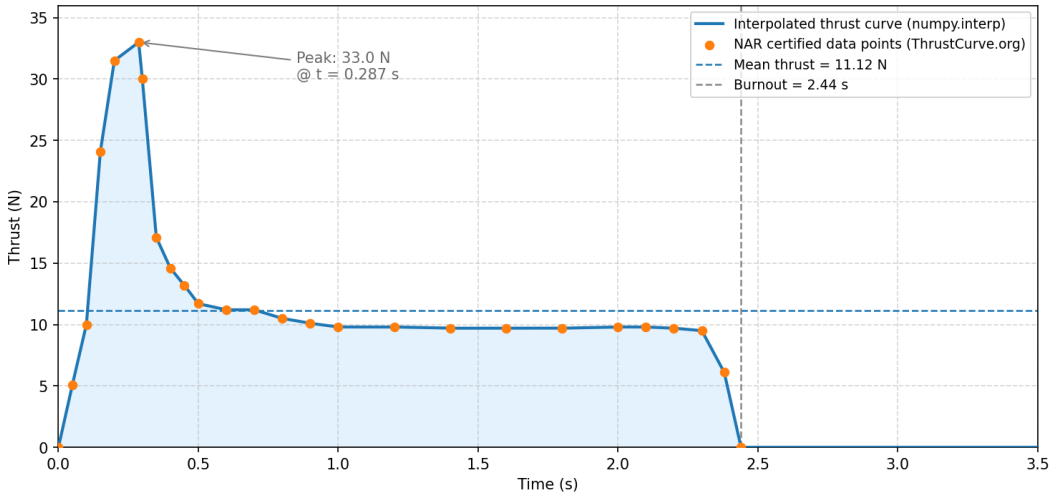

Figure 2: Estes® E12-4 Thrust Curve - NAR Certified Data (ThrustCurve.org). Peak = 33.0 N at $t = 0.29$ s, Mean = 11.12 N, Burn time = 2.44 s.

6.3 Baseline Closed-Loop Response

Figure 3 shows the baseline closed-loop attitude response to a 5° initial pitch offset, representing a realistic launch tip-off disturbance. The top panel shows that the true pitch angle is driven back toward vertical within approximately 0.3 seconds of ignition, with the Kalman filter estimate tracking closely during powered flight. The middle panel shows the corresponding angular rate,

which peaks near -45 deg/s immediately after correction begins before decaying smoothly as the controller damps rotational motion. The bottom panel shows gimbal deflection. The commanded angle briefly reaches the mechanical limit at the onset of correction, while the actual angle trails it slightly because the servo can only slew so fast, and both settle into small oscillatory commands as the vehicle stabilizes. After motor burnout at 2.44 s the control loop is disabled, and with the vehicle in near-freefall the Kalman estimate drifts on gyroscope integration alone. That post-burnout behavior is examined in Section 6.7, and it is distinct from the roll-axis, magnetometer-related drift of Appendix A.7.2.

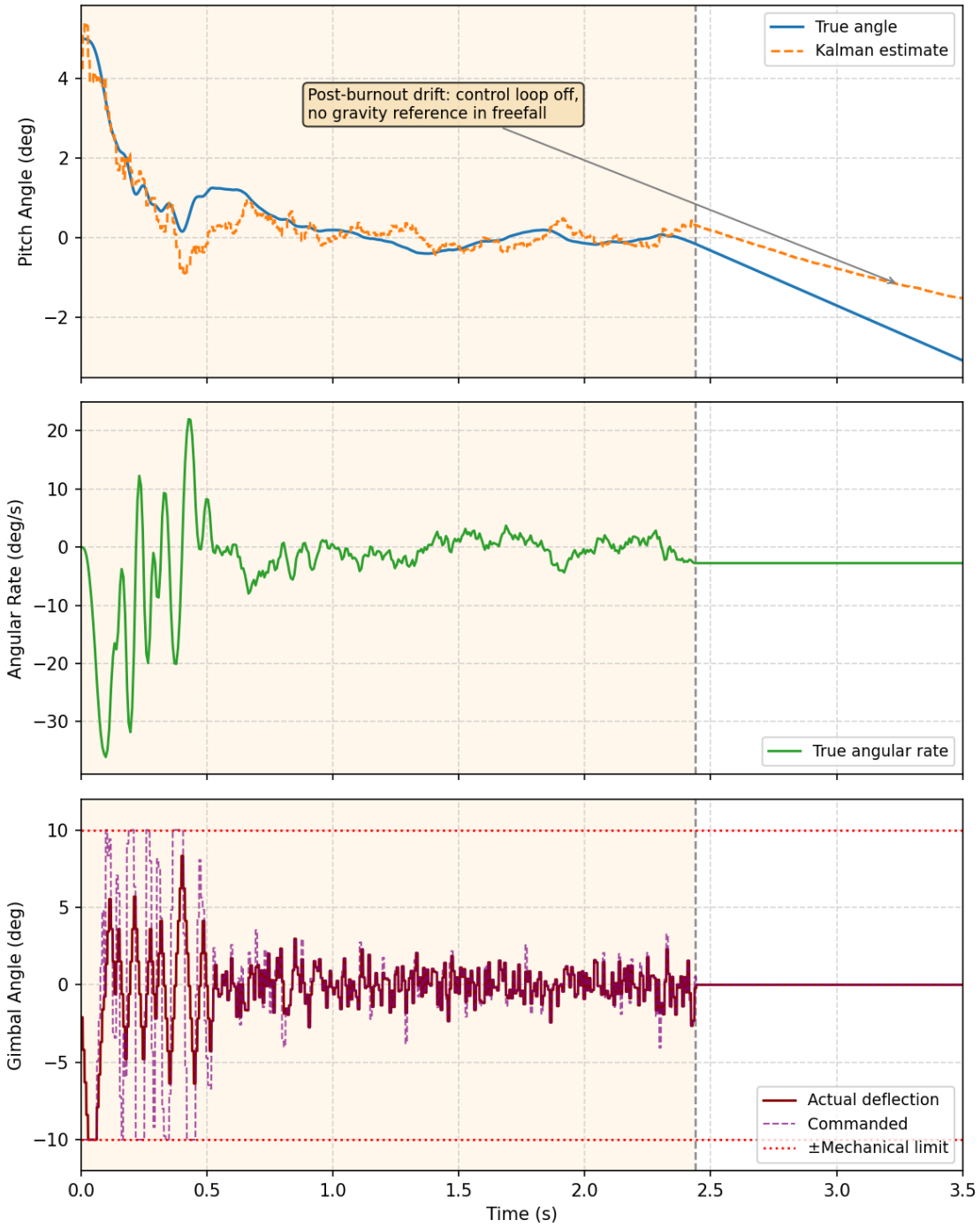

Figure 3: Attitude Stabilization Using TVC - Baseline Closed-Loop Response. Top: pitch angle (true vs. Kalman estimate). Middle: angular rate. Bottom: gimbal deflection with ±10° mechanical limits shown. Motor burn window shaded; post-burnout gyroscopic drift annotated.

6.4 Open-Loop vs. Closed-Loop Comparison

Figure 4 provides the most direct argument for the necessity of active thrust vector control in this finless configuration. Starting from the same 5° initial offset, the open-loop case - representing a rocket with no active stabilization - shows no tendency to return toward vertical. Because the vehicle is aerodynamically unstable in this finless configuration (Section 2.5), there is no restoring moment to correct the deviation; at the relatively low dynamic pressures and short burn duration modeled here, the destabilizing aerodynamic moment is not large enough to produce visible growth of the tilt angle over the burn window shown, but the same lack of a restoring moment means that, given more time, a larger initial disturbance, or higher dynamic pressure, this deviation would diverge rather than decay. The closed-loop case corrects the same disturbance within approximately 0.3 seconds and maintains near-zero pitch angle through burnout. This contrast illustrates that passive stability is entirely absent in this design, and that the PID-Kalman control architecture is solely responsible for maintaining a stable trajectory.

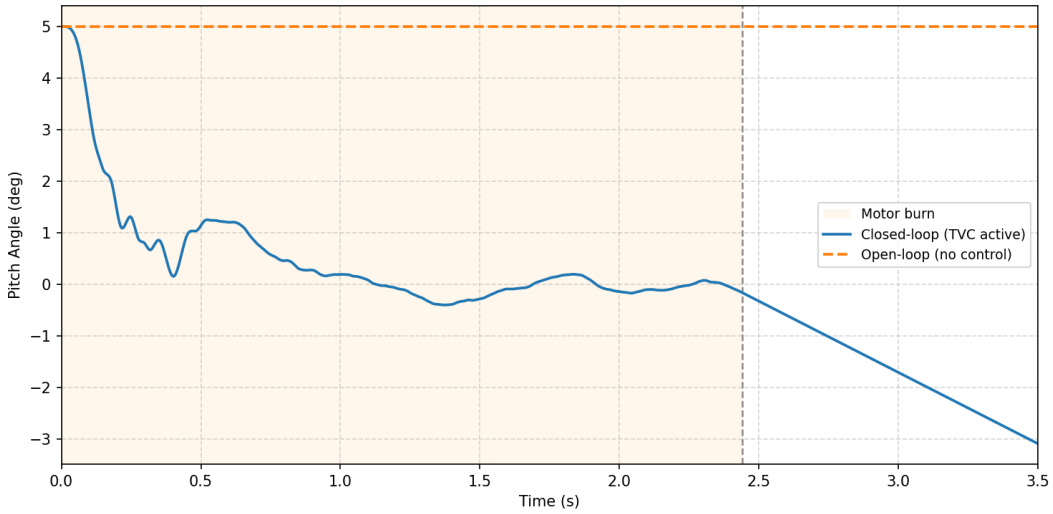

Figure 4: Open-Loop vs. Closed-Loop Attitude Response (Finless Configuration, $\theta_0 = 5°$). Without active control, the initial offset persists throughout the burn. Closed-loop TVC corrects the disturbance within ~0.3 s.

6.5 Disturbance Rejection Response

Figure 5 evaluates how well the system rejects a mid-flight disturbance. A torque impulse of 0.12 N·m applied for 50 ms at $t = 0.6$ s stands in for a sudden wind gust during powered ascent. The impulse drives the vehicle to a peak deviation of about 1.3°, after which the controller arrests the motion and brings it back toward the nominal trajectory, with recovery complete within 0.4 seconds. Within the range modeled here, the system tolerates this kind of aerodynamic disturbance comfortably.

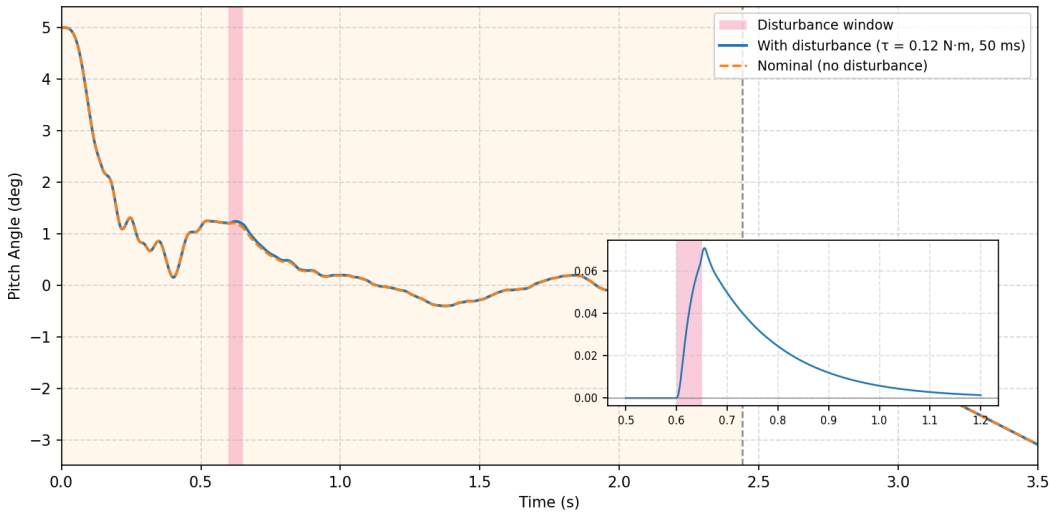

Figure 5: Disturbance Rejection Response Under TVC Control ($\tau = 0.12$ N·m, 50 ms at $t = 0.6$ s). Disturbance window highlighted. Nominal (no-disturbance) trace shown for comparison.

6.6 Control Effort and Actuator Utilization

Figure 6 shows how the controller turns attitude error into actuator motion, and confirms that the servo stays within its limits. In the top panel, the actual gimbal angle trails the commanded angle slightly during the initial correction, which is the servo's rate limiting at work as it slews to catch up. The bottom panel tracks cumulative actuator saturation, the fraction of the burn for which the command has been pinned at the $\pm 10°$ mechanical limit. Saturation is confined almost entirely to the initial tip-over correction ($t < 0.06$ s) and then decays, settling near 1.0% of the total burn. That this fraction stays so low matters, since a controller forced to saturate for much of the burn would have little authority left for further disturbances. Together these confirm that the PD gains suit the MG90S servo. Saturation is limited to the brief opening correction rather than running through the burn, and the system holds stability without sustained heavy actuator demand.

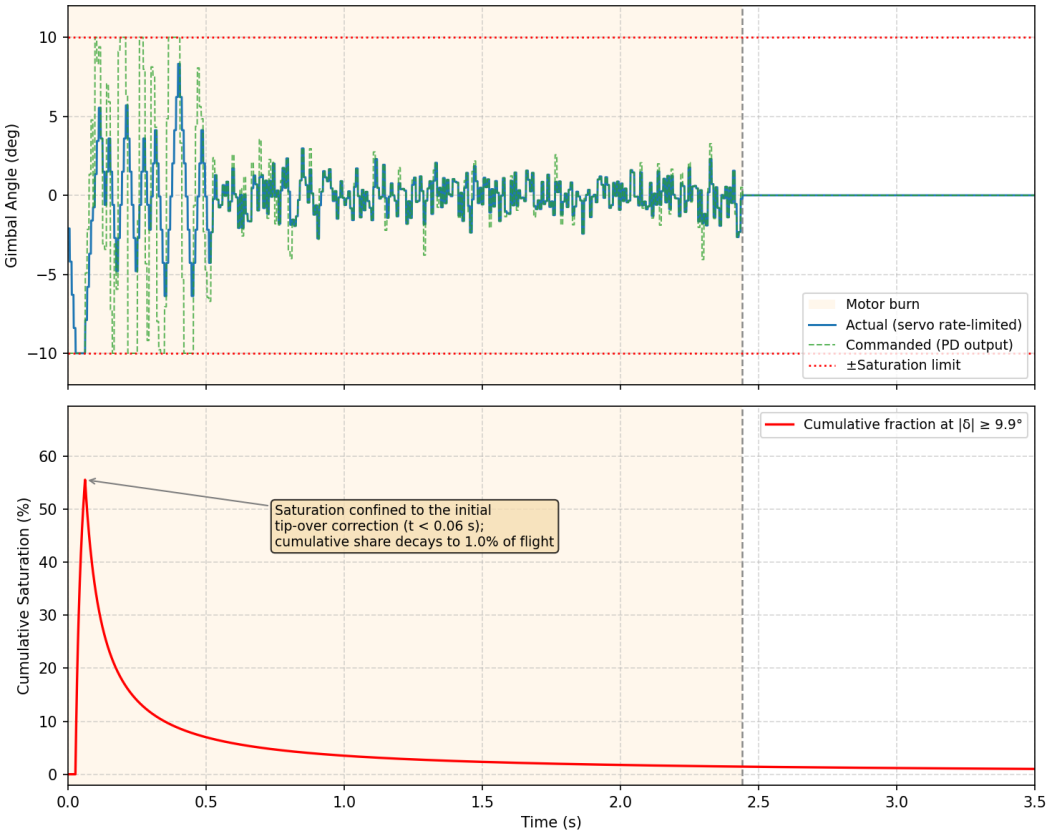

Figure 6: Control effort. Top: commanded versus achieved gimbal deflection. Bottom: cumulative actuator saturation as a fraction of burn time. Saturation is confined to the initial tip-over correction ($t < 0.06$ s) and settles near 1.0% of the burn.

6.7 Kalman Filter Performance

Figure 7 isolates the performance of the Kalman filter by comparing the raw accelerometer signal, the filtered attitude estimate, and the noise-free ground truth across the full simulation window. During powered flight, the raw accelerometer output is heavily corrupted by vibration-induced noise, with instantaneous errors exceeding ±4°. The Kalman estimate tracks the ground truth closely throughout the burn, suppressing high-frequency noise while preserving the low-frequency attitude trend. After burnout, the filtered estimate begins to diverge from ground truth. With the control loop disabled and the vehicle in freefall, the accelerometer no longer senses a usable gravity vector, so the filter falls back on gyroscope-only integration, which drifts over time. This is a separate effect from the roll-axis heading drift discussed in Appendix A.7.2, which stems from the MPU-6050's lack of a magnetometer (8) and applies only during powered flight. This post-burnout behavior is expected and does not affect the control system, which is disabled after motor shutdown.

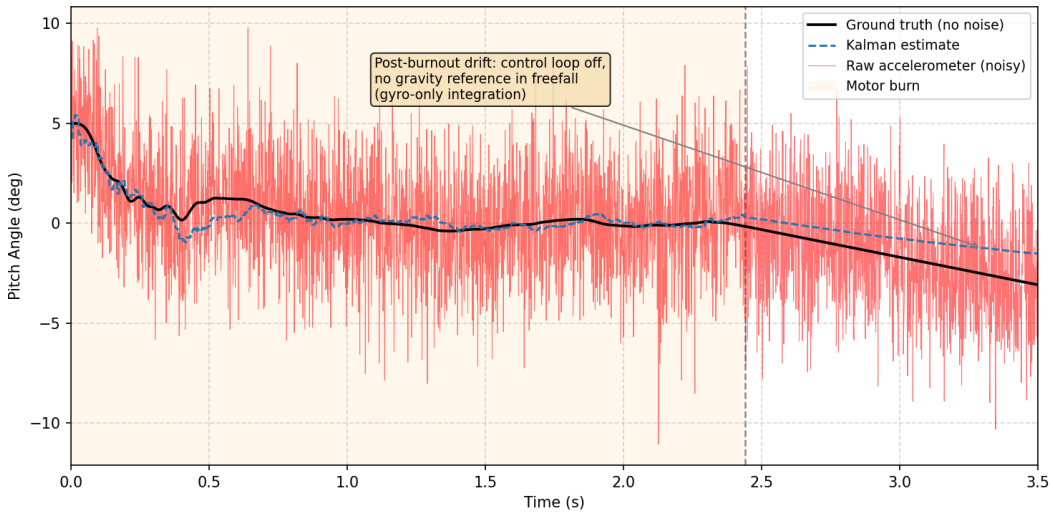

Figure 7: Kalman filter performance against the raw measurement (MPU-6050 noise model, Section 3.2). The filter suppresses the ±4° peak accelerometer noise during powered flight. Post-burnout gyroscopic drift is annotated.

6.8 TVC Corrective Torque Authority

Figure 8 plots the maximum corrective torque available from the TVC system throughout the motor burn, calculated using the relationship from Section 3.4.1:

$$\tau_{\max} = F_T \cdot r \cdot \sin(\delta_{\max}) \tag{eq20}$$

The torque peaks at 1.65 N·m near maximum thrust and holds steady at approximately 0.5 N·m during the sustained burn phase. Since the control authority margin remains substantial compared to a representative disturbance torque of 0.025 N·m, taken as an estimate of the aerodynamic torque a modest wind gust would impose on a vehicle of this size, stability is maintained throughout powered flight. The corrective authority profile mirrors the thrust curve, as expected from the governing equation. The collapse of available torque at burnout is why the flight software disables active control immediately after motor shutdown.

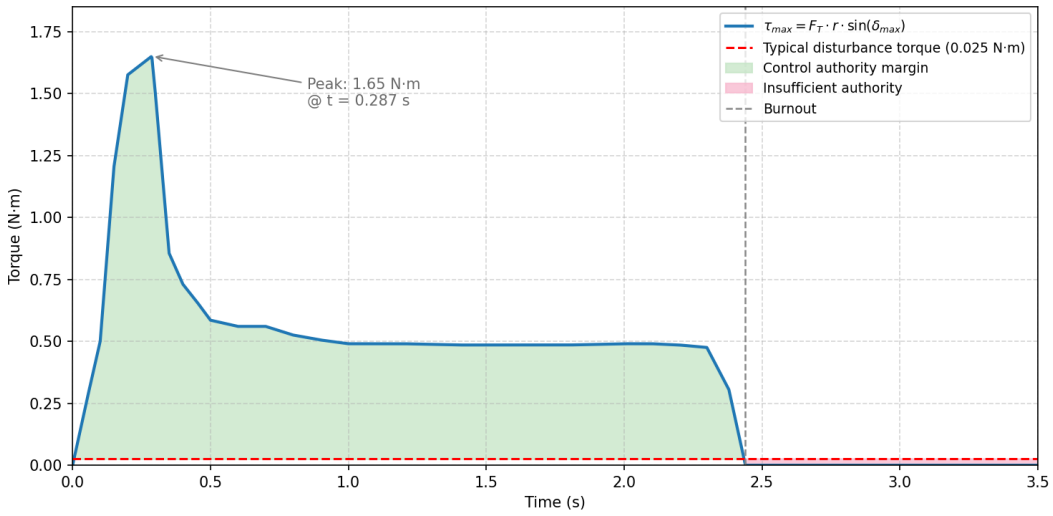

Figure 8: TVC Corrective Torque Authority vs. Time ($\tau_{\max} = F_T \cdot r \cdot \sin(\delta_{\max})$). Green shading: control authority margin above typical disturbance (dashed red). Burnout at 2.44 s marked; torque collapses immediately thereafter.

6.9 Parametric Sensitivity Analysis

Figure 9 examines how sensitive the pitch response is to three design parameters, namely motor thrust, the center-of-mass-to-nozzle distance, and the maximum gimbal angle. Each parameter was swept on its own while the others were held at their nominal values, with the baseline closed-loop simulation re-run from the same 5° initial offset for every setting, so that each panel isolates the effect of one parameter. Variations in motor thrust(±30%) and CoM-to-nozzle distance (±20%) caused negligible changes in recovery performance, confirming the control architecture's robustness to manufacturing tolerances and motor lot variation. Maximum gimbal angle had the greatest effect: stable recovery was maintained even when the limit was reduced to 6° or 8°, while increasing it to 12° produced a slightly more aggressive initial correction. This confirms that the 10° design choice offers adequate authority with a meaningful margin, and that stability is unlikely to be compromised by minor assembly deviations.

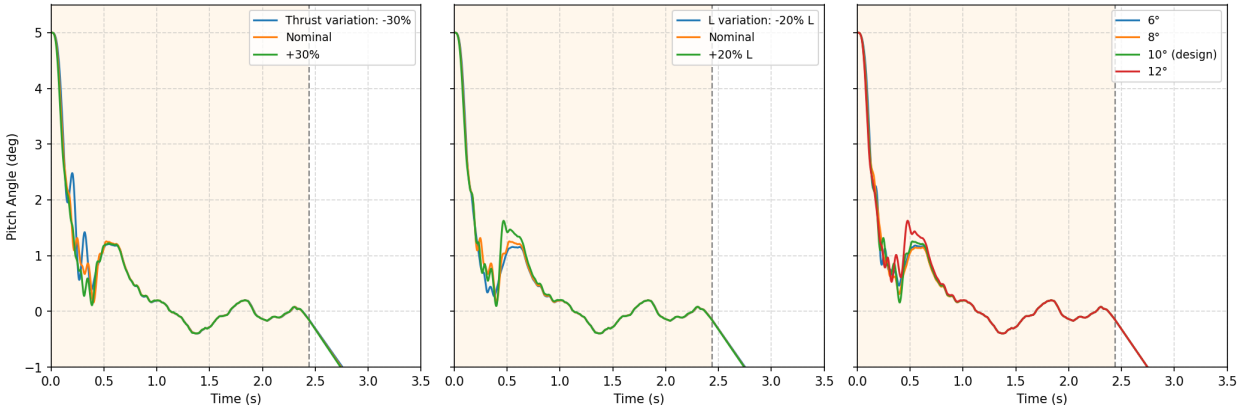

Figure 9: Parametric Sensitivity Analysis - Pitch Response ($\theta_0 = 5°$). (a) Motor thrust variation (±30%). (b) CoM-to-nozzle distance variation (±20%). (c) Maximum gimbal angle (6°, 8°, 10° design, 12°).

6.10 Controllability Boundary

Figure 10 maps the two-dimensional space of initial pitch angle and initial angular rate into recoverable and unrecoverable regions, providing a visual representation of the domain of controllability discussed analytically in Section 3.4. The green region indicates initial conditions from which the controller successfully recovers the vehicle during powered flight; the red region represents conditions from which divergence is unavoidable given the gimbal's physical limits. The boundary occurs at approximately 12°–15° of initial pitch angle depending on initial angular rate, consistent with the theoretical estimate derived in Section 3.4.2. This figure directly justifies the conservative launch detection thresholds implemented in Section 4. If the vehicle is tilted beyond this boundary at ignition, no amount of control effort can recover stable flight.

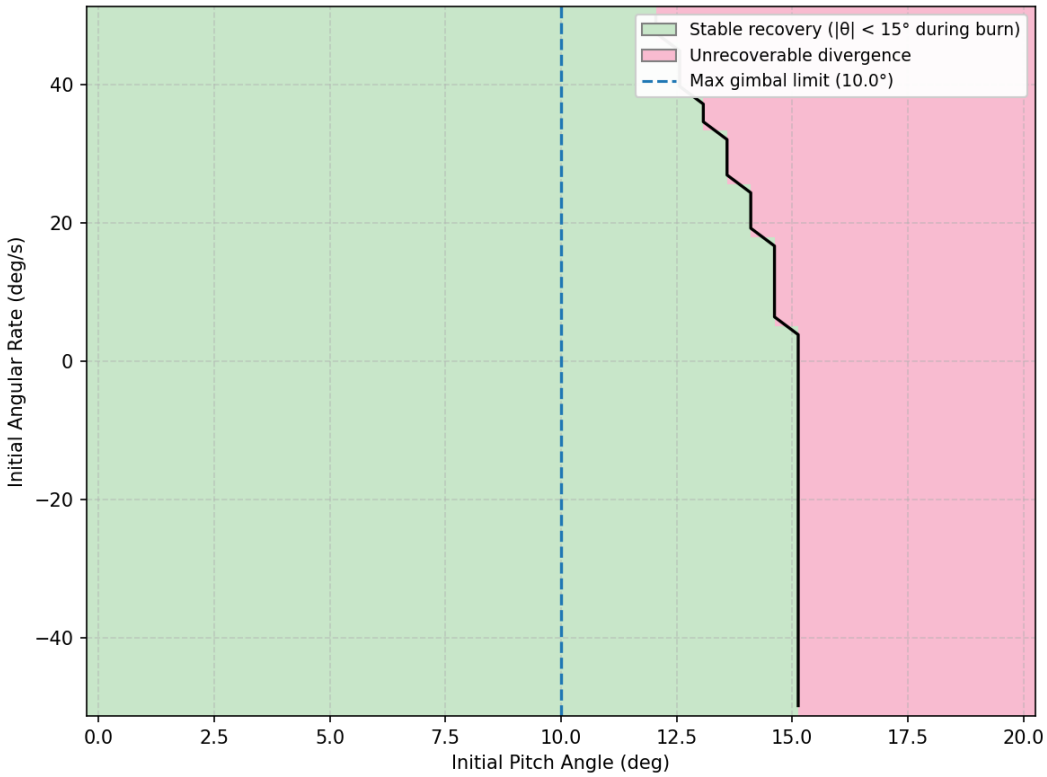

Figure 10: TVC controllability boundary in the plane of initial tilt θ_0 and initial angular rate $\dot{\theta}_0$. Green marks stable recovery ($|\theta| < 15°$ during the burn) and red marks unrecoverable divergence. The dashed vertical line is the maximum gimbal limit (10.0°).

6.11 Two-Axis Monte Carlo Simulation

Figure 11 provides the section's most thorough validation through 100 independent trials. These trials randomized initial pitch and yaw angles (0.5° to 6°), scaled thrust by ±8% to mimic motor manufacturing variance, and included IMU sensor noise at levels consistent with the MPU-6050 datasheet (8). In every trial, the rocket recovered successfully during powered flight and maintained stable orientation. The percentile envelope narrows quickly after the initial correction, which shows that performance stays consistent across the full range of randomized inputs. Individual traces still drift apart after burnout, since each trial integrates its own gyroscopic noise, but taken together the trials show that the control architecture holds up against realistic launch variability, as long as the

vehicle starts within the recoverable envelope of Section 6.10.

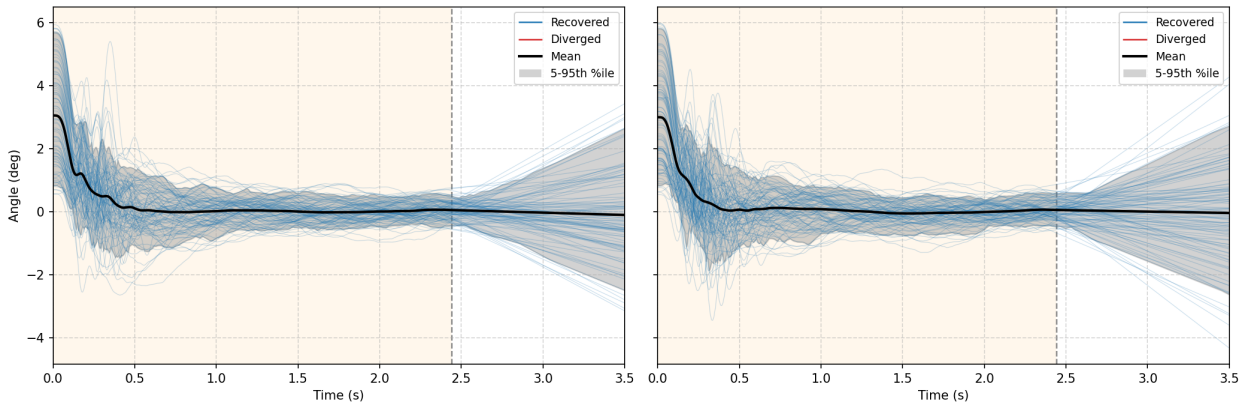

Figure 11: Two-axis Monte Carlo simulation under realistic launch variability (100 trials; $\theta_0 \in [0.5°, 6°]$, $T_{\text{scale}} \in [\pm 8\%]$, sensor noise included). Pitch (left) and yaw (right) axes are shown independently. All 100 trials recovered. The mean and 5th–95th percentile envelope are shown.

6.12 Discussion of Results and Limitations

Taken together, the simulations demonstrate consistent closed-loop stability across a wide range of initial conditions, disturbance magnitudes, and parameter variations. The control system respects actuator constraints, responds predictably to external inputs, and produces failure modes that align with the analytical limits derived in Section 3.4 rather than arising from control instability. The 100% Monte Carlo recovery rate within the tested envelope provides meaningful confidence in the architecture's robustness (Table 1).

The simulations do not capture all real-world effects. Aerodynamic nonlinearities at high angles of attack, structural vibration and aeroelastic coupling, meaning the feedback between airframe flexing and the aerodynamic loads that flexing creates, and launch rail dynamics are not modeled. Sensor noise models, while representative of the MPU-6050's published characteristics (8), cannot fully replicate the high-acceleration environment of an actual motor firing. These results should therefore be interpreted as validation of the system architecture and control design rather than a guarantee of flight performance. Physical validation using a 2-DOF static test stand-described as future work in Section 7-will provide the next level of verification.

6.13 Reproducibility and Data Availability

All simulation scripts, parameter files, and plotting code used to generate the figures in this section are available in the public GitHub repository referenced in Appendix A.6. Providing access to these materials allows the results to be independently verified and enables future refinement of the model.

7 Conclusion

This project began with a straightforward question: could a high school student, working independently, design and implement a functional thrust vector control system for a model rocket from

Table 1: Summary of Simulation-Based Performance Results

Metric	Value
Settling time, baseline 5° tip-off (Section 6.3)	~0.3 s
Peak angular deviation, mid-flight disturbance (Section 6.5)	1.3°
Recovery time, mid-flight disturbance (Section 6.5)	0.4 s
Peak corrective torque, τ_{max} (Section 6.8)	1.65 N·m
Sustained corrective torque, mid-burn (Section 6.8)	~0.5 N·m
Representative disturbance torque (Section 6.8)	0.025 N·m
Cumulative actuator saturation (Section 6.6)	~1.0% of burn
Maximum recoverable tilt angle, θ_{max} (Sections 3.4.2, 6.10)	12°–15°
Kalman filter noise reduction vs. raw accelerometer (Section 3.2)	70–80%
Monte Carlo recovery rate, 100 trials (Section 6.11)	100/100

the ground up? The answer, after years of iterative hardware design, self-directed learning, and rigorous analysis, is yes. The process of getting there is perhaps as significant as the result itself.

From a technical standpoint, this paper has demonstrated that thrust vector control is a viable stabilization mechanism at the hobby scale. The PID-Kalman control architecture, implemented on a Teensy® 4.1 microcontroller and driven by real-time MPU-6050 sensor data (8), produced stable closed-loop behavior across a range of simulated disturbance scenarios. Simulation results consistently showed that the system could arrest angular deviations, reject external disturbances, and operate within actuator limits without passive aerodynamic stabilization. The Monte Carlo trials tested that robustness under several sources of uncertainty at once, and showed that when the system did fail, it failed because of physical limits rather than any instability in the control loop.

It is important to be clear about the boundaries of these conclusions. The validation presented here is entirely simulation-based. Airspace restrictions prevented powered flight testing, which means that real-world phenomena-including structural vibration, aeroelastic coupling, launch rail dynamics, and high-acceleration sensor behavior-remain unverified. The rigid-body assumption underlying all simulations is a significant simplification, and the gap between simulated and actual flight performance is something only a real launch can close. This paper should therefore be read as a rigorous design and analysis study rather than a flight-validated system.

Several natural next steps follow from this work. A companion research study is currently underway to directly quantify the accuracy of the simulation framework developed here, using a custom 2-DOF static test stand to conduct live motor firings and compare measured angular error against simulation predictions using the Integral of Absolute Error as the primary performance metric. This work will identify where the simulation diverges from physical reality and trace that gap back to specific unmodeled phenomena such as actuator rate saturation, gimbal friction, and vibration noise. Beyond that, integrating a barometric pressure sensor would provide an independent altitude measurement for more robust apogee detection. On the control side, adaptive gain scheduling-where PID parameters shift as a function of thrust magnitude or estimated dynamic pressure-could meaningfully improve performance during the thrust decay phase where control authority diminishes most rapidly. Extending the platform to a multi-stage configuration would introduce additional complexity in both the state machine and the control architecture. Multi-stage thrust

vectoring has been built at the hobby scale before, notably the three-stage vehicle developed by BPS.Space (2), which makes it a demanding but logical progression of the current design.

More broadly, the goal of this project was never exclusively technical. Thrust vector control remains largely inaccessible in hobbyist and educational rocketry, not because the physics are beyond reach, but because the path from concept to implementation is poorly documented at the small scale. If this paper helps another student shortcut even a fraction of the trial and error that went into this system, it will have done exactly what it was meant to do.

A Flight Software Implementation Details

This appendix provides supporting technical material referenced throughout the paper, including flight software logic, control algorithms, and simulation implementations. These materials are included to improve reproducibility, transparency, and technical completeness while preserving the flow and readability of the main body.

All code presented here was developed specifically for this project and executed on the same computational and hardware platforms described in Sections 2–6.

A.1 State Machine Architecture Overview

Listing 1: High-level autonomous flight state machine (intended free-flight architecture; the altitude-based descent and recovery transitions are not exercised by the ground-constrained 2 DOF test-stand system).

```
if state == "Pad Idle":
    if accel_z > launch_threshold:
        state = "Powered Flight"

elif state == "Powered Flight":
    if vertical_velocity < 0 and time_since_launch > 0.75:
        state = "Apogee Detection"

elif state == "Apogee Detection":
    if velocity_magnitude < descent_threshold:
        state = "Descent"

elif state == "Descent":
    if altitude < safe_altitude:
        state = "Recovery"

elif state == "Recovery":
    if impact_detected:
        state = "Post-landing"
```

While simplified for clarity, this structure captures the core decision-making framework described in Section 4. Additional safeguards, timeouts, and fault-handling logic are implemented in parallel to prevent unsafe or undefined state transitions.

A.2 Apogee Detection Logic

The apogee logic shown below is part of the intended free-flight design rather than the ground-constrained system validated in this paper, and it is included to show how the descent handoff was meant to work. Rather than triggering on a single zero-crossing of vertical velocity, it would combine several sustained sensor trends to stay robust against noise, transient oscillation, and sensor bias.

Listing 2: Multi-condition apogee detection logic (intended free-flight design, not implemented on the test-stand hardware).

```
if vertical_velocity < negative_threshold for > 150 ms
   and altitude_rate < epsilon for > 300 ms
   and time_since_launch > 0.75 s
   and abs(pitch) < 15 deg and abs(yaw) < 15 deg:
       apogee_detected = True
```

A timeout safeguard would force a transition into descent mode if apogee were not detected within a maximum flight time, so that recovery logic still ran in the event of sensor failure or extreme deviation.

A.3 Discrete-Time PID Control Law

The PID control system operates independently on pitch and yaw axes using identical control structures. Each loop consumes filtered attitude estimates from the Kalman filter and outputs a commanded gimbal deflection angle. For the simulation results presented in Section 6, K_i was set to zero, so the simulated controller operated as a PD loop (Section 3.1); the integral term and its associated windup protection, shown below, remain implemented in the flight software for use in other flight regimes.

Listing 3: Discrete-time PID update law implemented on the flight computer.

```
error = desired_angle - measured_angle
integral += error * dt
derivative = (error - previous_error) / dt

output = Kp * error + Ki * integral + Kd * derivative
output = constrain(output, -max_deflection, max_deflection)

previous_error = error
```

Integral windup protection is enforced by limiting the accumulated integral term when the actuator saturates. This prevents excessive overshoot during recovery from large disturbances, as discussed in Section 3.4.

A.4 Kalman Filter for Attitude Estimation

Attitude estimation is performed using a discrete Kalman filter that fuses gyroscope angular velocity data with accelerometer-based tilt estimates. The filter mitigates high-frequency noise from vibration while correcting for long-term gyro drift. The prediction and update equations implemented in software follow the scalar Kalman formulation described in Section 3.2 directly; because each axis (pitch and yaw) is treated as an independent one-dimensional filter, no matrix algebra is required in the implementation.

A.5 Simulation Code and Plot Generation

All simulations presented in Section 6 were generated using Python 3 and the Matplotlib and NumPy libraries. Simulations were executed on a local development environment using Visual Studio Code. The simulation environment assumes a rigid-body model; therefore, structural flex and aeroelastic effects are not factored into the numerical integration.

The following simulation cases are included:

- Motor thrust curve (E12-4, NAR-certified data)
- Baseline closed-loop response (pitch angle, angular rate, gimbal deflection)
- Open-loop vs. closed-loop comparison
- Disturbance rejection response
- Control effort: commanded vs. achieved gimbal deflection
- Kalman filter performance vs. raw accelerometer
- TVC corrective torque authority vs. time
- Parametric sensitivity analysis (thrust, CoM offset, gimbal angle)
- Controllability boundary map
- Two-axis Monte Carlo simulation with randomized initial conditions

Each simulation shares a common structure: (1) definition of vehicle parameters, (2) numerical integration of rotational dynamics via RK4, (3) PID-based control input calculation at 150 Hz, (4) optional disturbance injection, and (5) data logging and figure generation. Figures are referenced directly in Section 6 and labeled consistently with their corresponding filenames.

A.6 Code Availability and Reproducibility

To support reproducibility and future development, all flight software and simulation code are provided in a public GitHub repository. This repository includes embedded flight software source files, Python simulation scripts, plot generation utilities, and configuration files.

https://github.com/MJDACKIW/Design_and_Validation_of_a_Thrust_Vector_Controlled_Model_Rocket_Using_PID-Kalman_Architecture/tree/main

A.7 Technical Disclaimers and Simulation Assumptions

This section outlines the physical simplifications and hardware constraints assumed during the design, control law development, and simulation of the TVC system. Acknowledging these factors is essential for evaluating the performance envelope and real-world reliability of the vehicle.

A.7.1 Rigid-Body Dynamics Assumption

All simulations in Section 6 treat the rocket airframe as a perfectly rigid body. In practice, aeroelasticity - where the airframe flexes under aerodynamic and inertial loads - can introduce high-frequency noise into IMU data, potentially leading to control-loop instability if the structural

resonance frequency approaches the PID sampling frequency. These structural-control interactions are not modeled in the current numerical integration.

A.7.2 Sensor Limitations and Roll Drift

The avionics use the MPU-6050 (8), which has no onboard magnetometer. The Kalman filter therefore estimates pitch and yaw with high fidelity, since those are the two axes the accelerometer can fix against gravity, but it has no absolute reference for heading about the roll axis. The PID loop actively stabilizes pitch and yaw, while roll, the rotation about the rocket's longitudinal axis, is left to drift under uncorrected gyroscope integration. For the short motor burns intended here, lasting 2 to 5 seconds, that drift stays negligible for flight stability, since roll does not affect the vehicle's ability to hold a vertical trajectory. It does, however, rule out precise heading-based navigation.

A.7.3 Aerodynamic Instability (Finless Configuration)

The decision to omit passive aerodynamic fins renders the vehicle inherently unstable, with the center of pressure (CoP) located forward of the center of mass (CoM). This creates a continuous inverted-pendulum scenario. The system's stability is entirely dependent on the active control loop and the gimbal's ability to maintain thrust-vector alignment with the vehicle axis. Any failure in the control software or mechanical gimbal response during the boost phase will result in immediate aerodynamic divergence.

A.7.4 Environmental Constants

Simulations assume a standard atmospheric model with constant air density at sea level. Variable wind shear, localized turbulence, and the change in atmospheric pressure with altitude are not factored into the disturbance rejection models. For a vehicle of this size and brief flight, these effects are minor, though they could introduce additional nonlinearities in a real flight.

References

1. Stuart, B., & McEnulty, J. (2023). *Overview of the SLS core stage thrust vector control system design* (Report No. AAS 23-152). NASA Marshall Space Flight Center. Presented at the 45th Annual AAS Guidance, Navigation and Control Conference, Breckenridge, CO. URL: https://ntrs.nasa.gov/citations/20230001023

2. BPS.Space (Barnard, J.). (2015–2023). *Model rocket thrust vector control development series* [Video series]. YouTube. URL: https://www.youtube.com/@BPSspace

3. TowerPro®. (n.d.). *MG90S metal gear servo - product specification*. URL: https://towerpro.com.tw/product/mg90s-3/

4. OpenRocket Development Team. (2023). *OpenRocket* (Version 23.09) [Computer software]. URL: https://openrocket.info/ (Originally developed by S. Niskanen, 2009; underlying thesis available at https://openrocket.sourceforge.net/thesis.pdf)

5. Barrowman, J. S. (1967). *The practical calculation of the aerodynamic characteristics of slender finned vehicles* [Master's thesis, Catholic University of America]. NASA Technical Reports Server. URL: https://ntrs.nasa.gov/citations/20010047838

6. Ziegler, J. G., & Nichols, N. B. (1942). Optimum settings for automatic controllers. *Transactions of the ASME, 64*(8), 759–765. DOI: https://doi.org/10.1115/1.4019264

7. Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Transactions of the ASME - Journal of Basic Engineering, 82*(1), 35–45. DOI: https://doi.org/10.1115/1.3662552

8. InvenSense. (2013). *MPU-6050 product specification* (Rev. 3.4). TDK InvenSense. URL: https://product.tdk.com/en/search/sensor/mortion-inertial/imu/info?part_no=MPU-6050

9. National Association of Rocketry. (2011). *Estes® E12 motor certification data*. ThrustCurve.org. URL: https://www.thrustcurve.org/motors/Estes/E12/

10. PJRC. (2023). *Teensy 4.1 development board*. URL: https://www.pjrc.com/store/teensy41.html